

BayesianFramework Cheat Sheet

Purpose: BayesianFramework makes it easier to build, train, and evaluate hierarchical Bayesian models for data containing multiple related entries. A model can contain parameters shared across all entries, called global parameters, as well as local parameters that vary between entries. Train uses training data to infer the global parameters, the local parameters of the training entries, and the relationships among local parameters. Test carries the inferred global parameters and local-parameter relationships forward to new entries, using them to infer new local parameters either from the training results alone or by incorporating observations from the new entries. ScoreTrain and ScoreTest evaluate how well the resulting model predictions match the data. Save preserves the inferred model state so that potentially expensive training or testing does not need to be repeated, while Load restores that state for further testing, scoring, or analysis. The user defines the mathematical model and likelihoods, while the framework handles the Bayesian inference and organization needed to train, test, score, save, and reuse the model.

Dependencies: jax, numpyro, numpy

Installation: Install in editable mode with `pip install -e "/path/to/BayesianFramework"` then import with `from BayesianFramework import BayesianFramework`. Editable installation means updates pulled into the repository are available without reinstalling. If using VS Code and the import is not recognized by autocomplete, add `"python.analysis.extraPaths": ["/path/to/BayesianFramework"]` to settings.json.

Conventions

`arg{}` - dictionary whose values have no entry dimension; used for one entry's data, local parameters, or model output, and for global parameters or other unbatched named values.

`arg{[]}` - dictionary containing arrays spanning multiple entries or samples.

`arg[]` - 1D sequence.

`Fn(...)` -> `value` - function argument or method together with the value it returns.

`A | B` - either input/return type A or input/return type B, depending on behavior.

Italic arguments - accept None as a valid input.

Main Functions

BayesianFramework(modelDataFull{[]}, ModelFn(globalParams{*GlobalParamFn*(dataShapes{*GlobalParamFn*)} -> globalParams{*choleskyConcentration*, rngSeed) -> BayesianFramework: Creates a framework object. **modelDataFull** contains the full dataset. **ModelFn** evaluates one entry. **localParamNames** optionally defines standardized entry-level parameters; when multiple locals are present, their correlation is learned with an LKJ prior controlled by **choleskyConcentration**, a positive number. **GlobalParamFn** optionally receives a dictionary of full-data array shapes and defines model-facing global parameters. **rngSeed** is an optional nonnegative integer that initializes the persistent RNG state. Stochastic operations advance this state; supplying **rngSeed** to a later call resets it before that operation. **Returns:** a new BayesianFramework object.

Train(trainIndices[], TrainLikelihoodFn(globalParams{*TrainLikelihoodFn*}, localParams{[]}, data{[]}, modelOut{[]}), numWarmup, numSamples, num_chains, acceptProb, dense_mass, medianSamples, printSummary, rngSeed, saveLocals, saveModelOut) -> MCMC: Fits the entries selected by **trainIndices** using NUTS MCMC. **TrainLikelihoodFn** defines the likelihood from the globals, locals, selected data, and model output. **numWarmup**, **numSamples**, **num_chains**, and **medianSamples** control MCMC; **acceptProb** is a probability; **dense_mass**, **printSummary**, **saveLocals**, and **saveModelOut** are booleans. **rngSeed** optionally resets the persistent RNG state before this call. **saveLocals** stores transformed training-local posterior samples; **saveModelOut** caches model outputs. **Returns:** the NumPyro MCMC object.

Test(testIndices[], TestLikelihoodFn(globalParams{*TestLikelihoodFn*}, localParams{[]}, data{[]}, modelOut{[]}), nGlobalSamples, numSamples, num_chains, numWarmup, acceptProb, dense_mass, medianSamples, printSummary, rngSeed, saveModelOut) -> None | MCMC[]: Evaluates the trained model on **testIndices**. **nGlobalSamples=None** uses the median training-global posterior state; a positive integer samples that many training-global states. Without **TestLikelihoodFn**, no test MCMC is run: **numSamples** local samples are drawn per global state and stored for later scoring. With **TestLikelihoodFn**, one test MCMC is run per global state; **numSamples** is samples per chain and **num_chains** is the chain count. **numWarmup**, **acceptProb**, **dense_mass**, and **medianSamples** apply only to MCMC testing. **printSummary** prints MCMC summaries only. **rngSeed** optionally resets the persistent RNG state before this call. **saveModelOut** optionally caches test model outputs. **Returns:** None without MCMC, otherwise a list of NumPyro MCMC objects.

ScoreTrain(ScoreFn(modelOut{*ScoreFn*}, data{*ScoreFn*}) -> score{*ScoreFn*}, nSamples, rngSeed) -> scores{[]}: Scores training trajectories. **ScoreFn evaluates one entry and may return a scalar, array, or dictionary. **nSamples=None** scores all available joint training-posterior samples; otherwise that many posterior samples are selected using **rngSeed** or the persistent RNG. Saved model outputs are used when available; otherwise outputs are regenerated from saved posterior parameters. **Returns:** scores with leading posterior-sample and training-entry dimensions.**

ScoreTest(ScoreFn(modelOut{*ScoreFn*}, data{*ScoreFn*}) -> score{*ScoreFn*}, nSamples, rngSeed) -> scores{[]}: Scores the most recent test state. **nSamples=None scores all available local samples; otherwise that many local samples are selected for every stored global sample using **rngSeed** or the persistent RNG. Saved model outputs are used when available; otherwise outputs are regenerated from saved test globals and locals. **Returns:** scores with leading global-sample, local-sample, and test-entry dimensions.**

Save(fileName, saveTrainLocals, saveTestLocals, saveTrainModelOut, saveTestModelOut, saveAllData) -> None: Writes framework state to a compressed NumPy .npz archive. Training globals and training Cholesky samples are always saved. **saveTrainLocals** and **saveTestLocals** default to True; saved test locals include their associated test-global and Cholesky state. **saveTrainModelOut**, **saveTestModelOut**, and **saveAllData** default to False. Functions are not serialized.

BayesianFramework.Load(fileName, ModelFn, GlobalParamFn, modelDataFull) -> BayesianFramework: Loads framework state from a .npz file. **ModelFn** and optional **GlobalParamFn** are supplied explicitly rather than deserialized; their names are checked against the saved metadata and mismatches produce warnings. **modelDataFull** must be supplied unless the file was saved with **saveAllData=True**. **Returns:** the reconstructed BayesianFramework object.